JobControl

Distributed Job Scheduler

Internship Project – Submission Report

Prepared By: Aadarsh Jaiswal

Registration Number: RA2311030010076

University: SRM Institute of Science and Technology

GitHub Repository: https://github.com/aadarshjaiswal640/Job-Scheduler

Submission Date: July 3, 2026

Abstract

This report presents JobControl, a distributed job scheduling system designed and built as part of an internship assignment. The system addresses the fundamental challenge of reliably executing background tasks in a concurrent, multi-tenant environment.

The implementation consists of three primary components: a FastAPI REST API server exposing 40+ endpoints for managing organizations, projects, queues, and jobs; a React single-page application dashboard providing real-time visibility into job execution, worker status, and queue health; and a background worker service that polls a PostgreSQL database, atomically claims jobs using FOR UPDATE SKIP LOCKED, and executes them concurrently using a thread pool.

Key features include priority-based job dispatch, three configurable retry strategies (exponential, linear, fixed), a dead letter queue for exhausted jobs, rule-based AI failure analysis, WebSocket real-time updates, and a multi-tenant organizational hierarchy. The system is built on a PostgreSQL database with nine tables and uses JWT authentication with bcrypt password hashing.

Table of Contents

1. Introduction

Modern software systems rely heavily on background processing — sending emails, generating reports, processing payments, resizing images, and executing data pipelines. These workloads cannot block the user-facing request-response cycle; they must be deferred, queued, and executed asynchronously by worker processes.

Managing these background tasks at scale introduces several challenges: ensuring each job runs exactly once, handling transient failures gracefully, providing visibility into execution status, and enforcing resource limits so that one runaway job does not starve all others.

JobControl is a full-stack distributed job scheduler built to address these challenges. It provides a complete platform for defining job queues, submitting jobs (immediate, scheduled, or recurring), monitoring their execution in real time, and recovering failed jobs through configurable retry policies and a dead letter queue.

The system is designed with multi-tenancy at its core — multiple organizations can operate independently within a shared deployment, each with their own projects, queues, and workers. Authentication is handled through JWT tokens, and all resource access is gated by organization membership checks.

2. Problem Statement

Background job processing is a foundational component of any production web application. Without a reliable job scheduler, developers must resort to ad-hoc solutions such as cron jobs on individual servers, which suffer from:

- No visibility — no centralized view of what is running, what has failed, or why
- No retry logic — a failed job is simply lost unless manually re-run
- No concurrency control — multiple server instances can execute the same job twice
- No prioritization — urgent jobs wait in the same queue as low-priority batch tasks
- No multi-tenancy — different teams share infrastructure with no isolation boundary

This project solves: how to build a reliable, observable, and fair job scheduling system that handles failures gracefully, prevents duplicate execution, and provides real-time insight into system health — while remaining deployable as a single coherent application.

3. Objectives

1. Design and implement a multi-tenant job scheduling system with a clear organizational hierarchy (Organization → Project → Queue → Job).

2. Build a REST API with complete CRUD for all resources and purpose-built endpoints for job lifecycle management.

3. Implement a background worker that polls the database and executes jobs concurrently using a thread pool.

4. Enforce atomicity in job claiming using PostgreSQL's FOR UPDATE SKIP LOCKED to prevent duplicate execution.

5. Support three job types — immediate execution, one-shot scheduled, and recurring (cron expression).

6. Configure retry policies — exponential, linear, and fixed delay strategies with per-queue configuration.

7. Implement a dead letter queue for jobs that exhaust all retries, with retry and restore operations.

8. Build a React dashboard with real-time metrics, execution charts, activity feed, and queue summary.

9. Implement JWT authentication with access and refresh tokens and bcrypt password hashing.

10. Document the system with architecture diagrams, ER diagram, API documentation, design decisions, and this submission report.

4. Technology Stack

Table 1: Technology Stack

Layer	Technology	Version	Purpose
Backend API	FastAPI	0.115.5	REST API framework with automatic OpenAPI docs
ASGI Server	Uvicorn	0.32.1	High-performance async server
ORM	SQLAlchemy	2.0.36	Database abstraction layer
Database	PostgreSQL	Latest	Primary data store; FOR UPDATE SKIP LOCKED
DB Driver	psycopg2-binary	2.9.10	PostgreSQL adapter for Python
Auth	python-jose / PyJWT	—	JWT token creation and verification
Password Hashing	passlib[bcrypt]	1.7.4	Secure bcrypt password hashing
Data Validation	Pydantic v2	2.10.3	Request/response schema validation
Settings	pydantic-settings	2.6.1	Environment variable configuration
Migrations	Alembic	1.14.0	Database schema versioning (installed)
Frontend	React 18	18	Component-based UI library
Language	TypeScript	5	Type-safe JavaScript
Build Tool	Vite	Latest	Fast frontend bundler with HMR
Styling	Tailwind CSS	4	Utility-first CSS framework

UI Components	Radix UI / shadcn/ui	Latest	Accessible, unstyled UI components
Data Fetching	TanStack Query	Latest	Cache-aware API data management
Charts	Recharts	2.15.2	React charting library
Router	Wouter	3.3.5	Lightweight React router
Forms	react-hook-form + zod	Latest	Form state management and validation

5. System Architecture

JobControl follows a three-tier architecture: client tier (React SPA), application tier (FastAPI + background worker), and data tier (PostgreSQL).

5.1 Architecture Overview

```
CLIENT TIER
  React 18 SPA — Vite — TypeScript — Tailwind CSS
  TanStack Query (HTTP caching) · Wouter (routing)
  WebSocket connection for real-time events
          | HTTPS REST (Bearer JWT)       | WebSocket /ws
APPLICATION TIER — FastAPI 0.115.5 + Uvicorn ASGI
  Routers: auth · organizations · projects · queues
           jobs · workers · dashboard · dlq · health
  Modules: auth.py · deps.py · access.py · websocket.py
  ┌── Background Worker (Daemon Thread) ──────────────┐
  │  Poll 3s · Heartbeat 30s · ThreadPoolExecutor(20) │
  │  FOR UPDATE SKIP LOCKED · Retry · DLQ             │
  └───────────────────────────────────────────────────┘
          | SQLAlchemy 2.0  pool_size=10+5
DATA TIER — PostgreSQL
  users · organizations · organization_members · projects
  queues · jobs · job_executions · job_logs · workers
  dead_letter_queue · activity_events
```

5.2 Component Boundaries

Client Tier: The React SPA runs entirely in the browser. TanStack Query manages all server state. A WebSocket hook (use-live-updates.ts) maintains a persistent connection and calls queryClient.invalidateQueries() on receiving events, triggering dashboard refetches.

Application Tier: FastAPI handles all REST requests on the main async event loop. Nine router groups cover all resources. The background worker daemon thread uses its own SQLAlchemy session and connection pool (pool_size=5) to avoid competing with the API pool (pool_size=10).

Data Tier: PostgreSQL stores all system state in nine tables. All primary keys are UUID v4. The FOR UPDATE SKIP LOCKED SQL primitive enables atomic job claiming without an external message broker.

6. Database Design

The JobControl schema consists of nine tables organized around a four-level tenant hierarchy: Organization → Project → Queue → Job. All primary keys are UUID v4 values.

Table 2: Database Tables Summary

Table	Purpose	Key Relationships
users	User accounts; bcrypt passwords	Owns orgs; member of org_members
organizations	Top-level tenant boundaries	Owned by user; has projects
organization_members	User ↔ Org join table with role	FK: org_id, user_id; unique(org_id, user_id)
projects	Groups of queues within an org	FK: org_id (CASCADE)
queues	Named job queues with policy config	FK: project_id (CASCADE)
jobs	Individual work units	FK: queue_id (CASCADE), worker_id (SET NULL)
job_executions	Per-attempt audit records	FK: job_id (CASCADE)
job_logs	Structured log lines per job	FK: job_id (CASCADE)
workers	Worker process registrations + heartbeat	Referenced by jobs
dead_letter_queue	Snapshots of exhausted jobs	Soft refs to jobs, queues
activity_events	System-wide event log	Soft refs only (append-only)

6.1 Key Design Choices

• **UUID Primary Keys:** Eliminates sequential ID guessing in URLs; supports horizontal scaling without central coordination.

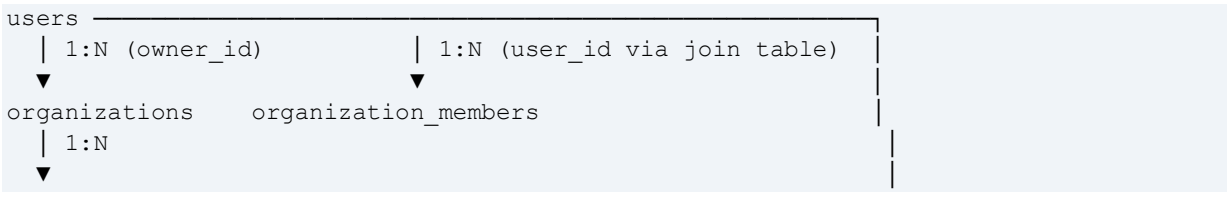
• **Cascade Delete Chain:** Deleting an organization cascades to projects → queues → jobs → executions → logs, keeping the database clean.

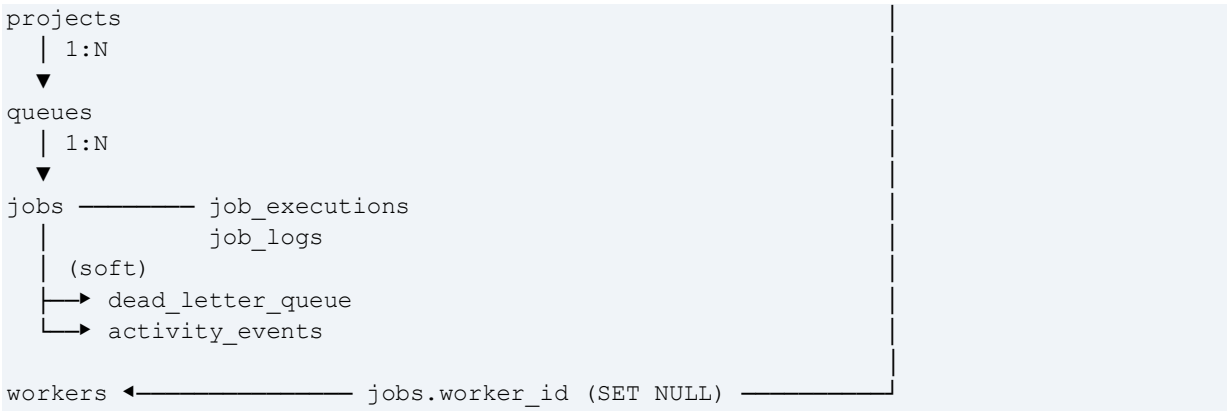
• **Soft References in DLQ & Activity Events:** dead_letter_queue and activity_events outlive the records they reference, preserving audit history after queue deletion.

• **Composite Indexes:** ix_jobs_queue_status (queue_id, status) serves the worker's claim query; ix_jobs_status_scheduled (status, scheduled_at) serves retry promotion.

• **JSON Payload Column:** jobs.payload is JSON type — arbitrary job input without schema migrations as job types evolve.

6.2 Entity-Relationship Diagram





7. Backend Architecture

The FastAPI backend is organized into five core modules and nine router files.

7.1 Core Modules

main.py: App factory; lifespan (DB init + worker thread start); CORS; router mounts; WebSocket endpoint at /ws

models.py: 9 SQLAlchemy ORM models with UUID primary keys, relationships, cascade rules, and composite indexes

auth.py: hash_password, verify_password (bcrypt); create_access_token, create_refresh_token, decode_token (PyJWT HS256)

config.py: pydantic_settings.BaseSettings; reads DATABASE_URL, SESSION_SECRET from environment variables

database.py: SQLAlchemy engine (pool_size=10, max_overflow=20), SessionLocal factory, Base declarative class

deps.py: get_current_user FastAPI dependency (HTTPBearer → JWT decode → DB lookup); require_admin

access.py: require_project, require_queue, require_job, require_dlq_entry — all resolve resource AND verify org membership

websocket.py: WebSocketManager — set of active connections; broadcast fan-out with automatic dead-connection pruning

7.2 Request Lifecycle

```
HTTP Request
    ↓
HTTPBearer (FastAPI security)
    ↓
get_current_user (deps.py)
    → decode JWT → DB lookup → User object
    ↓
access.py helper (require_queue / require_job / etc.)
```

```
    → load resource → assert org membership
    ↓
Route handler → business logic → DB write → JSON response
```

8. Frontend Architecture

The frontend is a Vite-bundled React 18 TypeScript single-page application with 11 pages, a shared Shell layout, and approximately 60 shadcn/ui component primitives.

8.1 Route Map

/login, /register — Public authentication pages

/dashboard — Stat cards, 24h area chart, activity feed, queue summary table

/organizations — Organization list + create dialog

/organizations/:id — Org detail: members, invite, projects list

/projects/:id — Project detail: queues list, create queue

/queues/:id — Queue detail: job list, metrics, pause/resume

/jobs — Job Explorer: paginated, filterable, searchable

/jobs/:id — Job detail: metadata, execution history, log viewer

/workers — Worker fleet: status, CPU/memory, heartbeat

/dlq — Dead Letter Queue: browse, retry, restore, delete

/settings — User settings page

8.2 State Management

TanStack Query manages all server state. Each page calls generated API hooks (e.g., useGetDashboardStats()) that cache responses, refetch in the background, and expose isLoading, data, and error states. There is no global Redux/Zustand store.

Authentication state is managed by AuthProvider (React Context), storing JWT tokens in localStorage. The ProtectedRoute component redirects unauthenticated users to /login.

The WebSocket hook (use-live-updates.ts) maintains a persistent /ws connection and calls queryClient.invalidateQueries() on event receipt, keeping the dashboard current without polling.

9. Worker Architecture

The background worker runs as a daemon thread started in main.py's lifespan context. Being a daemon thread, it is automatically terminated when the main process exits. It uses its own SQLAlchemy engine and session to avoid competing with the API's connection pool.

9.1 Execution Loop

```
run_worker() starts
  │
  ├─ register_worker() → INSERT workers row → get WORKER_ID
  │
  └─ while True:  (POLL_INTERVAL = 3 seconds)
       │
       ├─ Heartbeat every 30s
       │    └─ UPDATE workers SET last_heartbeat, cpu_usage, memory_usage
       │
       └─ claim_and_dispatch_jobs()
            ├─ SELECT non-paused queues ORDER BY priority DESC
            └─ For each queue:
                 ├─ Count running jobs; skip if at concurrency limit
                 ├─ Promote retry jobs (scheduled_at ≤ now → queued)
                 ├─ Promote scheduled one-shot jobs → queued
                 └─ SELECT queued jobs LIMIT slots
                    FOR UPDATE SKIP LOCKED
                    ORDER BY priority DESC, created_at ASC
                       → UPDATE status=running → executor.submit(...)
```

9.2 Job Execution Thread

Each job runs in a separate thread from the ThreadPoolExecutor (max_workers=20). The thread:
1. Opens its own SQLAlchemy session (pool_size=2)
2. Locks the job row with with_for_update()
3. Creates a JobExecution record (attempt_number = retry_count + 1)
4. Simulates work: time.sleep(random.uniform(0.5, 4.0))
5. On success (80%): sets job.status = 'completed'
6. On failure (20%): evaluates retry eligibility; re-schedules or moves to DLQ
7. Closes session and disposes engine

10. Authentication & Security

10.1 JWT Token Design

Token	Algorithm	TTL	Purpose
Access Token	HS256	30 minutes	Authenticate API requests
Refresh Token	HS256	7 days	Obtain new access tokens

Tokens carry a 'type' claim ('access' or 'refresh'). The get_current_user dependency rejects any token where type != 'access', preventing refresh tokens from being used directly as API credentials. Passwords are hashed with bcrypt (12 rounds) via passlib, automatically embedding a salt — making rainbow table attacks infeasible.

10.2 Authorization Model

All resource access walks the tenant hierarchy:
• require_project(project_id) — checks org membership for the project's org

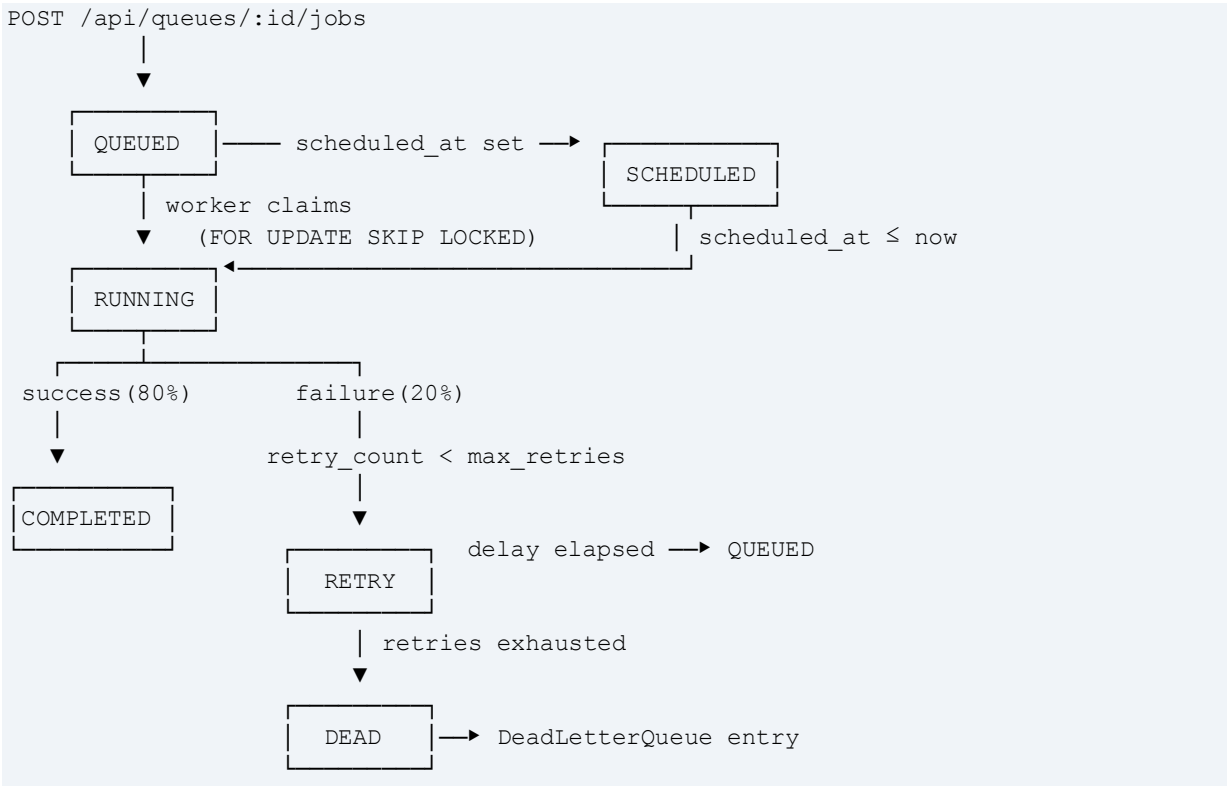
• require_queue(queue_id) — resolves queue → project → org → membership
• require_job(job_id) — delegates to require_queue
• require_dlq_entry(dlq_id) — delegates to require_queue

This single-point-of-enforcement pattern in access.py ensures authorization logic cannot be accidentally omitted in any new route.

11. Job Lifecycle

Table 3: Job Status Values

Status	Description	Next States
queued	Ready for worker pickup	running, failed (cancelled)
scheduled	Waiting for scheduled_at to pass	queued (promoted by worker)
running	Currently executing in thread	completed, retry, dead
retry	Waiting for retry delay to elapse	queued (promoted by worker)
completed	Successfully finished	queued (if manually retried)
failed	Failed or cancelled by user	queued (if manually retried)
dead	Exhausted all retries; in DLQ	queued (if restored from DLQ)



12. Queue Management

Queues are the central scheduling unit in JobControl. Each queue is independently configurable:

- **Name / Description:** Human-readable identifiers for the queue
- **Priority:** Integer (default 5); determines dispatch order when multiple queues are non-empty
- **Concurrency:** Integer (default 5); maximum simultaneous running jobs for this queue
- **Retry Policy:** strategy (exponential/linear/fixed), max_retries, retry_interval_seconds
- **Paused Flag:** When true, worker skips this queue entirely; all pending jobs remain preserved

13. Retry Policies

When a job fails, the worker calculates a retry delay, sets status to 'retry', and sets scheduled_at = now + delay. The worker's next poll cycle promotes jobs with scheduled_at ≤ now back to 'queued'.

Table 4: Retry Strategy Formula

Strategy	Formula	Delay at attempt 1	Delay at attempt 3
exponential (default)	min(base × 2^retry_count, 3600)	60s	240s
linear	base × (retry_count + 1)	60s	180s
fixed	base	60s	60s

Exponential backoff is the default and is most appropriate for transient failures (network errors, rate limits) because it reduces pressure on downstream services during incident recovery. The cap at 3600 seconds (1 hour) prevents indefinitely long delays.

14. Dead Letter Queue

When retry_count reaches max_retries, the worker sets job.status = 'dead', writes an ai_failure_summary, creates a dead_letter_queue row (snapshot of job_name, queue_name, payload, failure_reason), and writes a job_failed activity event.

Operation	Endpoint	Behavior
List	GET /api/dlq	Paginated; scoped to user's accessible queues
Inspect	GET /api/dlq/{id}	Full entry including payload snapshot
Retry	POST /api/dlq/{id}/retry	Creates a NEW job with same payload; deletes DLQ entry
Restore	POST /api/dlq/{id}/restore	Resets ORIGINAL job to 'queued'; deletes DLQ entry
Delete	DELETE /api/dlq/{id}	Permanently removes the DLQ entry

14.1 AI Failure Summary

The ai_failure_summary field is populated by deterministic rule-based keyword matching — not an external LLM API. Error substrings are mapped to human-readable explanations:

Error Pattern	Generated Summary
timeout	Job timed out during execution. Consider increasing the timeout limit...
connection / network	Network connectivity issue detected. Retrying may resolve this...
memory / oom	Memory exhaustion detected. Reduce payload size or increase limits...
permission / auth	Authorization failure. Check credentials...
not found / 404	Resource not found. The job referenced a resource that no longer exists.
rate limit / 429	Rate limit exceeded. Implement exponential backoff...
simulated	Simulated execution failure for demonstration purposes.
(default)	Execution failed with error: <first 120 chars>. Review job payload.

15. Concurrency Model

15.1 Queue-Level Concurrency

Each queue has a concurrency setting (default 5). Before claiming jobs, the worker counts jobs with status='running' for that queue. If running >= concurrency, the queue is skipped. A queue with concurrency=1 acts as a serial queue — essential for jobs accessing shared resources.

15.2 Atomic Job Claiming — FOR UPDATE SKIP LOCKED

```
SELECT * FROM jobs
WHERE queue_id = :queue_id AND status = 'queued'
ORDER BY priority DESC, created_at ASC
LIMIT :available_slots
FOR UPDATE SKIP LOCKED;
```

FOR UPDATE acquires row-level locks on the selected rows. SKIP LOCKED skips rows already locked by another transaction. This allows multiple worker instances to run simultaneously without coordination — each claims a disjoint set of jobs. This is a well-established PostgreSQL idiom for building job queues without an external broker.

16. REST API Catalogue

Table 5: API Endpoint Inventory

Method	Path	Description
POST	/api/auth/register	Register new user; auto-creates personal org
POST	/api/auth/login	Authenticate; returns access + refresh tokens
POST	/api/auth/refresh	Exchange refresh token for new access token

GET	/api/auth/me	Return current user profile
GET	/api/organizations	List user's organizations
POST	/api/organizations	Create organization
GET	/api/organizations/{id}	Get organization detail
PATCH	/api/organizations/{id}	Update organization name/description
DELETE	/api/organizations/{id}	Delete organization (owner only)
GET	/api/organizations/{id}/members	List organization members
POST	/api/organizations/{id}/invite	Invite user by email
DELETE	/api/organizations/{id}/members/{mid}	Remove member
GET	/api/organizations/{id}/projects	List org projects
POST	/api/organizations/{id}/projects	Create project
GET	/api/projects/{id}	Get project
PATCH	/api/projects/{id}	Update project
DELETE	/api/projects/{id}	Delete project (cascade)
GET	/api/projects/{id}/queues	List project queues
POST	/api/projects/{id}/queues	Create queue with retry policy
GET	/api/queues/{id}	Get queue with job counts
PATCH	/api/queues/{id}	Update queue configuration
DELETE	/api/queues/{id}	Delete queue (cascade)
POST	/api/queues/{id}/pause	Pause queue
POST	/api/queues/{id}/resume	Resume queue
GET	/api/queues/{id}/metrics	Queue performance metrics
GET	/api/queues/{id}/jobs	List queue jobs
POST	/api/queues/{id}/jobs	Create a job
POST	/api/queues/{id}/jobs/batch	Batch create jobs
GET	/api/jobs	List all accessible jobs (paginated, filtered)
GET	/api/jobs/{id}	Get job detail
DELETE	/api/jobs/{id}	Delete job
POST	/api/jobs/{id}/retry	Reset job to queued (retry_count=0)
POST	/api/jobs/{id}/cancel	Cancel job → failed
GET	/api/jobs/{id}/logs	Get job log entries
GET	/api/jobs/{id}/executions	Get execution attempt history
GET	/api/workers	List all registered workers
GET	/api/workers/{id}	Get worker detail
GET	/api/dashboard/stats	Aggregate system statistics
GET	/api/dashboard/metrics	Time-series metrics (1h/6h/24h/7d/30d)
GET	/api/dashboard/activity	Recent activity events
GET	/api/dashboard/queue-summary	All queues summary table
GET	/api/dlq	List DLQ entries (paginated)
GET	/api/dlq/{id}	Get DLQ entry
POST	/api/dlq/{id}/retry	New job from DLQ entry
POST	/api/dlq/{id}/restore	Restore original job from DLQ
DELETE	/api/dlq/{id}	Delete DLQ entry
GET	/api/healthz	Health check (DB + worker count)
WS	/ws	WebSocket real-time event stream

17. Dashboard & Monitoring

17.1 Stat Cards (8 metrics)

Metric	Source Field
Running Jobs	dashboard/stats.running_jobs
Queued Jobs	dashboard/stats.queued_jobs
Active Workers	dashboard/stats.active_workers
Success Rate (%)	dashboard/stats.success_rate
Failed Jobs	dashboard/stats.failed_jobs
Avg Execution	dashboard/stats.avg_execution_ms
Throughput / min	dashboard/stats.throughput_per_minute
DLQ Count	dashboard/stats.dlq_count

17.2 Execution Throughput Chart

A Recharts AreaChart displays completed and failed job counts over the last 24 hours (up to 24 time buckets). Two area series use Tailwind CSS custom properties for colors, automatically adapting to dark/light mode. The chart is powered by GET /api/dashboard/metrics?period=24h.

17.3 Activity Feed

The 10 most recent activity_events are displayed with event-type icons: green check (completed), red X (failed), rotation arrow (retried), server icon (worker events). Each entry shows the description and a formatted timestamp.

17.4 Health Check

GET /api/healthz executes SELECT 1 against the database and returns:
{ "status": "ok", "database": "healthy", "worker_count": 1 }
This endpoint is suitable for use as a container health probe or load balancer check.

18. Execution Logs

JobControl maintains two complementary log structures for every job.

18.1 Structured Job Logs (job_logs table)

Every significant lifecycle event creates a job_log entry with level (info/warning/error) and a message. Events logged include: job created, execution started (attempt N), job completed (duration ms), execution failed (error), retry scheduled (attempt n/max, delay), job cancelled, job manually retried, re-queued from DLQ.

18.2 Execution Records (job_executions table)

Each attempt creates a job_execution row with attempt_number (1-based), status (running/completed/failed), started_at, completed_at, duration_ms, error_message, and output. This provides a machine-readable audit trail of all retry attempts.

19. Testing

All testing was performed manually against the running application. The TESTING.md document defines 25 test cases across authentication, organizations, queues, jobs, workers, retry logic, DLQ, dashboard, and security.

19.1 Key Test Cases

ID	Area	Input	Expected Result
AUTH-001	Auth	POST /auth/register (valid)	201; access_token + refresh_token; personal org created
AUTH-002	Auth	POST /auth/register (duplicate email)	400 — Email already registered
AUTH-003	Auth	POST /auth/register (password < 8 chars)	400 — Password too short
AUTH-005	Auth	POST /auth/login (wrong password)	401 — Invalid credentials
AUTH-008	Auth	Bearer: 'invalidtoken'	401 — Invalid or expired token
JOB-001	Job	POST /jobs (immediate); wait 10s	status=completed (or retry at 20% failure rate)
JOB-002	Job	POST /jobs (scheduled, 30s ahead)	status=scheduled; transitions to running after 30s
JOB-004	Job	POST /jobs/{id}/cancel	status=failed, error_message='Cancelled by user'
RET-001	Retry	Queue: exponential, base=10s	Delays: 10s, 20s, 40s for attempts 1, 2, 3
DLQ-001	DLQ	Queue max_retries=0; job fails	DLQ entry with job_name, failure_reason, payload
DLQ-002	DLQ	POST /dlq/{id}/retry	New job created; DLQ entry deleted
SEC-001	Security	Cross-org queue access	403 Forbidden
SEC-002	Security	Use refresh token as Bearer	401 — type != 'access'

19.2 Future: Automated Test Setup

Recommended automated test suites:

• pytest (backend): tests/test_auth.py, test_organizations.py, test_jobs.py, test_worker.py, test_dlq.py

• Vitest + React Testing Library (frontend): auth-provider, dashboard, job-explorer components

• httpx for async API integration tests against a test database

20. Design Decisions

Decision	Choice	Primary Rationale
Backend framework	FastAPI 0.115.5	Auto-docs, type safety, async/WebSocket, clean DI
Database	PostgreSQL	FOR UPDATE SKIP LOCKED; ACID; JSON columns; no broker needed
ORM	SQLAlchemy 2.0 (sync)	Clean models; thread-safe; pool_pre_ping resilience
Authentication	JWT HS256 + bcrypt	Stateless, simple to implement, secure password hashing
Frontend framework	React 18 + TypeScript	Component model; TanStack Query caching; type safety
Build tool	Vite	Sub-second HMR; fast production builds
UI components	shadcn/ui + Tailwind CSS v4	Accessible Radix primitives; full style control
Worker model	Daemon thread + ThreadPoolExecutor	Single-process simplicity; no external broker
Job claiming	FOR UPDATE SKIP LOCKED	Atomic; broker-free; horizontally scalable
Authorization	Centralized access.py	Single enforcement point; easy to audit
AI failure summary	Rule-based keyword matching	No external API cost or latency; deterministic
Multi-tenancy	Org → Project → Queue hierarchy	Enterprise-grade isolation; mirrors Temporal/Bull

21. Future Enhancements

21.1 High Priority

Enhancement	Description
Recurring Job Rescheduling	cron_expression stored but worker doesn't re-schedule. Add croniter to calculate next_run.
Automated Test Suite	No pytest or Vitest tests exist. Required for production readiness.
Stale Running Job Reaper	Detect jobs stuck in 'running' (worker crash) and reset to 'queued'.
WebSocket Authentication	Validate JWT on /ws connect to prevent unauthorized event stream access.

21.2 Medium Priority

Enhancement	Description
Docker / docker-compose	Containerize API, worker, and frontend for reproducible deployment.

Alembic Migration Scripts	Generate versioned migration files for schema changes.
JWT Refresh Token Rotation	Single-use refresh tokens that are invalidated on use.
RBAC within Organizations	Read-only member roles enforced at the resource level.
Rate Limiting	slowapi or nginx-level rate limiting on auth endpoints.

22. Conclusion

JobControl demonstrates a complete, production-oriented distributed job scheduling system built with modern Python and TypeScript tooling.

- **Reliability:** PostgreSQL FOR UPDATE SKIP LOCKED ensures each job is claimed exactly once; atomic transactions prevent partial state updates; the worker loop recovers from transient failures automatically.
- **Observability:** The dashboard provides real-time visibility into execution throughput, worker health, queue state, and failure rates. Every job has a full execution history and structured log trail.
- **Configurability:** Each queue is independently configurable with priority, concurrency limits, and retry strategy. Individual jobs can override queue-level settings.
- **Recoverability:** The dead letter queue captures all exhausted jobs with their payloads intact, enabling one-click retry or restore. Rule-based AI analysis provides actionable failure summaries.
- **Security:** JWT authentication with bcrypt password hashing, centralized authorization checks, and type-discriminated tokens provide a solid security foundation.

The multi-tenant organizational hierarchy scales naturally from a single developer's personal organization to an enterprise deployment with dozens of teams. Future work priorities include automated test coverage, recurring job rescheduling, Docker containerization, and WebSocket authentication — all building directly on the solid foundation established by this implementation.

23. References

[1] FastAPI Documentation — https://fastapi.tiangolo.com/

[2] SQLAlchemy 2.0 Documentation — https://docs.sqlalchemy.org/en/20/

[3] PostgreSQL FOR UPDATE SKIP LOCKED — https://www.postgresql.org/docs/current/sql-select.html

[4] PyJWT Documentation — https://pyjwt.readthedocs.io/

[5] Passlib (bcrypt) Documentation — https://passlib.readthedocs.io/

[6] React Documentation — https://react.dev/

[7] TanStack Query Documentation — https://tanstack.com/query/latest

[8] Recharts Documentation — https://recharts.org/

[9] Tailwind CSS Documentation — https://tailwindcss.com/docs/

[10] shadcn/ui Documentation — https://ui.shadcn.com/

[11] Wouter Documentation — https://github.com/molefrog/wouter

[12] Vite Documentation — https://vitejs.dev/

[13] Pydantic v2 Documentation — https://docs.pydantic.dev/latest/

[14] Python concurrent.futures — https://docs.python.org/3/library/concurrent.futures.html

[15] RFC 7519 — JSON Web Tokens — https://tools.ietf.org/html/rfc7519

24. Appendix

Appendix A: Environment Variables

Variable	Required	Default	Description
DATABASE_URL	Yes	—	PostgreSQL connection string (postgresql://...)
SESSION_SECRET	Yes	supersecretkey-change-in-production	JWT signing secret; must be changed in production

Appendix B: Default Configuration Values

Setting	Default	Location
Access token TTL	30 minutes	config.py
Refresh token TTL	7 days	config.py
JWT algorithm	HS256	config.py
Worker poll interval	3 seconds	worker/main.py
Worker heartbeat interval	30 seconds	worker/main.py
Thread pool max workers	20	worker/main.py
API DB pool_size	10	database.py
API DB max_overflow	20	database.py
Queue default priority	5	models.py
Queue default concurrency	5	models.py
Queue default max retries	3	models.py
Queue default retry interval	60 seconds	models.py
Queue default retry strategy	exponential	models.py
Job success simulation rate	80%	worker/main.py

Appendix C: Assignment Requirement Mapping

Requirement	Status	Evidence
Distributed Job Scheduler	✓Implemented	worker/main.py — poll/claim/execute

		loop
Job Queue CRUD	✅Implemented	routers/queues.py
Immediate Job Type	✅Implemented	job_type=immediate + worker dispatch
Scheduled Job Type	✅Implemented	scheduled_at + worker promotion
Recurring Job Type (cron)	⚠ Partial	cron_expression stored; rescheduling loop pending
Priority-Based Dispatch	✅Implemented	Queue priority + job priority ordering
Retry Policies (3 strategies)	✅Implemented	calculate_retry_delay() in worker/main.py
Dead Letter Queue	✅Implemented	dead_letter_queue table + /api/dlq router
DLQ Retry / Restore	✅Implemented	POST /api/dlq/{id}/retry and /restore
Concurrency Control	✅Implemented	Per-queue concurrency + FOR UPDATE SKIP LOCKED
Worker Architecture	✅Implemented	Daemon thread + ThreadPoolExecutor(20)
Worker Heartbeat	✅Implemented	30-second heartbeat cycle
Execution Logging	✅Implemented	job_logs + job_executions tables
JWT Authentication	✅Implemented	HS256 access (30m) + refresh (7d) tokens
Multi-tenant Organizations	✅Implemented	Org → Project → Queue → Job hierarchy
REST API (40+ endpoints)	✅Implemented	9 router groups
WebSocket Real-time Updates	✅Implemented	/ws endpoint + WebSocketManager
React Dashboard	✅Implemented	Stats, charts, activity, queue table
AI Failure Summary	✅Implemented	Rule-based ai_failure_summary()
Automated Tests	❌Not implemented	Listed as Future Enhancement
Docker / docker-compose	❌Not implemented	Listed as Future Enhancement